

IMS Server Deployment Guide

Inventory Management System — Production Deployment

Version: 1.0

Prepared for: Client Onboarding

Date: May 2026

Overview

This guide walks through the end-to-end process of deploying the Inventory Management System (IMS) to a production server so it runs reliably 24/7. Each step covers the tools required, the exact commands to run, and key decisions to make along the way. The guide assumes a Linux VPS (Ubuntu 22.04 recommended) and a custom domain name.

#	Step	Description
1	Choose a VPS	Select and provision a cloud server
2	Install Dependencies	Python, pip, Nginx, PostgreSQL
3	Upload the Application	Transfer code to the server
4	Gunicorn (WSGI Server)	Replace dev server with production server
5	Nginx (Reverse Proxy)	Route traffic and handle SSL termination
6	Domain & SSL	Point your domain and get a free certificate
7	systemd Service	Keep the app alive 24/7 automatically
8	Database Upgrade	Migrate SQLite to PostgreSQL for scale
9	Backups & Security	Automated backups and access controls

1 Choose a VPS (Virtual Private Server)

Pick a cloud provider and spin up your server

A VPS is a virtual machine rented from a cloud provider. It runs 24/7 and gives you a fixed IP address and full Linux control. For IMS, a basic 1-2 vCPU / 2 GB RAM instance is sufficient to start.

Recommended Providers

Provider	Cost	Notes
DigitalOcean Droplet	~\$6/mo	Simple UI, great docs, Sydney region available

Vultr	~\$6/mo	Fast SSD, good AU latency
Hetzner	~€4/mo	Excellent value, EU-based
AWS EC2 (t3.small)	Pay-per-use	Enterprise-grade, more complex setup
Linode / Akamai	~\$5/mo	Reliable, developer-friendly

Minimum Recommended Specs

- OS: Ubuntu 22.04 LTS (64-bit)
- RAM: 2 GB (1 GB minimum)
- Storage: 25 GB SSD
- vCPUs: 1-2
- Bandwidth: 1 TB/month (standard for most providers)

Tip

Once provisioned, note the server's public IP address — you'll use it throughout this guide.

2 Install Dependencies

SSH into your server and install required packages

Connect to your server via SSH, then install Python, pip, and required system libraries.

Connect via SSH

Terminal

```
ssh root@YOUR_SERVER_IP
```

Update System and Install Packages

bash

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y python3 python3-pip python3-venv
sudo apt install -y nginx postgresql postgresql-contrib
sudo apt install -y certbot python3-certbot-nginx
```

Create a Dedicated App User (recommended)

Running the application as a non-root user improves security.

bash

```
sudo adduser --disabled-password --gecos '' imsapp
sudo usermod -aG sudo imsapp
```

3 Upload the Application

Transfer your code from your local machine to the server

There are two main ways to get your application code onto the server.

Option A — Git (recommended)

If your project is in a Git repository (e.g. GitHub, GitLab), clone it directly on the server:

bash

```
cd /var/www
sudo git clone https://github.com/YOUR_USER/ims-app.git
sudo chown -R imsapp:imsapp /var/www/ims-app
```

Option B — SCP (direct file copy)

Run this from your local machine:

Local Terminal

```
scp -r /path/to/ims-app root@YOUR_SERVER_IP:/var/www/ims-app
```

Set Up Python Virtual Environment

bash

```
cd /var/www/ims-app
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install gunicorn
```

Note

Make sure your requirements.txt is up to date before uploading. Run 'pip freeze > requirements.txt' locally first.

4

Gunicorn — WSGI Production Server

Replace Flask's dev server with a production-grade WSGI server

Flask's built-in development server is single-threaded and not suitable for production. Gunicorn handles multiple concurrent requests safely and is the standard Python WSGI server.

Test Gunicorn Manually

bash

```
cd /var/www/ims-app
source venv/bin/activate
gunicorn --workers 3 --bind 0.0.0.0:8000 app:app
```

Visit http://YOUR_SERVER_IP:8000 — if the app loads, Gunicorn is working. Press Ctrl+C to stop it (systemd will manage it permanently in Step 7).

Worker Count Rule of Thumb: Workers = (2 x CPU cores) + 1

CPU Cores	Recommended Workers
1	3

2	5
4	9

5 Nginx — Reverse Proxy

Route external web traffic to Gunicorn and handle static files

Nginx sits in front of Gunicorn, handling HTTPS termination, serving static files efficiently, and forwarding dynamic requests to your app.

Create Nginx Site Configuration

bash

```
sudo nano /etc/nginx/sites-available/ims-app
```

Paste the following configuration:

nginx.conf

```
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;
    location / {
        proxy_pass      http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
    location /static {
        alias /var/www/ims-app/static;
        expires 30d;
    }
}
```

Enable and Test the Configuration

bash

```
sudo ln -s /etc/nginx/sites-available/ims-app /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx
```

Note

Replace 'yourdomain.com' with your actual domain name in the config above.

6 Domain Name & SSL Certificate

Point your domain to the server and enable HTTPS for free

A custom domain and HTTPS are essential for professional deployment. Certbot from Let's Encrypt provides free, auto-renewing SSL certificates.

DNS Setup

- Log in to your domain registrar (e.g. GoDaddy, Namecheap, Cloudflare)
- Create an A Record pointing yourdomain.com to YOUR_SERVER_IP
- Create a CNAME Record pointing www to yourdomain.com
- Wait 5-30 minutes for DNS propagation

Install SSL Certificate with Certbot

bash

```
sudo certbot --nginx -d yourdomain.com -d www.yourdomain.com
```

Certbot will automatically modify your Nginx config to enable HTTPS and set up a cron job to auto-renew the certificate before it expires.

Verify Auto-Renewal

bash

```
sudo certbot renew --dry-run
```

Security

After this step, your app will be accessible at <https://yourdomain.com> with a valid SSL certificate. HTTP traffic is automatically redirected to HTTPS.

7 systemd Service — 24/7 Uptime

Configure the OS to manage the app as a background service

systemd is the Linux service manager. By registering IMS as a systemd service, it will start automatically on server boot and restart itself if it ever crashes.

Create the Service File

bash

```
sudo nano /etc/systemd/system/ims-app.service
```

Paste the following:

ims-app.service

```
[Unit]
Description=IMS Gunicorn Application
After=network.target
[Service]
User=imsapp
Group=www-data
WorkingDirectory=/var/www/ims-app
Environment='PATH=/var/www/ims-app/venv/bin'
ExecStart=/var/www/ims-app/venv/bin/gunicorn \
    --workers 3 \
```

```

    --bind unix:/var/www/ims-app/ims.sock \
    app:app
Restart=always
RestartSec=5
[Install]
WantedBy=multi-user.target

```

Enable and Start the Service

bash

```

sudo systemctl daemon-reload
sudo systemctl enable ims-app
sudo systemctl start ims-app
sudo systemctl status ims-app

```

Confirm

The status command should show 'active (running)'. If there are errors, check logs with: `sudo journalctl -u ims-app -n 50`

8 Database Upgrade — SQLite to PostgreSQL

Optional but recommended for multi-user production use

SQLite is fine for development and single-user use. For concurrent access (multiple users, background jobs), PostgreSQL is the recommended upgrade.

When to Upgrade

Scenario	SQLite OK?	Recommendation
Single user, light use	Yes	Keep SQLite
2-5 concurrent users	Marginal	Migrate to PostgreSQL
5+ users or API integrations	No	PostgreSQL required

PostgreSQL Setup Commands

bash

```

# Create database and user
sudo -u postgres psql
CREATE DATABASE imsdB;
CREATE USER imsuser WITH PASSWORD 'strongpassword';
GRANT ALL PRIVILEGES ON DATABASE imsdB TO imsuser;
\q
# Install Python PostgreSQL driver
pip install psycopg2-binary
# Migrate existing data
pip install pgloader
pgloader sqlite:///inventory.db postgresql://imsuser:strongpassword@localhost/imsdb

```

9 Backups and Security

Protect your data with automated backups and access controls

Automated Daily Backups

Create a backup script and schedule it with cron to run every night at 2 AM.

backup.sh

```
#!/bin/bash
# /var/www/ims-app/backup.sh
DATE=$(date +%Y-%m-%d)
BACKUP_DIR=/var/backups/ims
mkdir -p $BACKUP_DIR
# SQLite backup
cp /var/www/ims-app/inventory.db $BACKUP_DIR/inventory-$DATE.db
# Remove backups older than 30 days
find $BACKUP_DIR -name '*.db' -mtime +30 -delete
echo 'Backup complete: '$DATE
```

bash

```
chmod +x /var/www/ims-app/backup.sh
crontab -e
# Add this line:
0 2 * * * /var/www/ims-app/backup.sh >> /var/log/ims-backup.log 2>&1
```

Security Hardening Checklist

Control	Action
Firewall (UFW)	sudo ufw allow 22 && sudo ufw allow 80 && sudo ufw allow 443 && sudo ufw enable
SSH Key Auth	Disable password login in /etc/ssh/sshd_config: PasswordAuthentication no
Fail2Ban	sudo apt install fail2ban — blocks IPs after repeated failed login attempts
HTTPS Only	Certbot handles this automatically; verify with: curl -I http://yourdomain.com
App Secret Key	Set FLASK_SECRET_KEY as an environment variable — never hardcode in source
DB Passwords	Use strong random passwords; store in environment variables or a secrets vault

Quick Command Reference

Task	Command
Check app status	sudo systemctl status ims-app
Restart app	sudo systemctl restart ims-app
View live logs	sudo journalctl -u ims-app -f
Reload Nginx	sudo systemctl reload nginx

Renew SSL certificate	<code>sudo certbot renew</code>
Pull latest code	<code>cd /var/www/ims-app && git pull && sudo systemctl restart ims-app</code>

You're Ready to Go Live

Once all 9 steps are complete, your Inventory Management System will be running in production — secured with HTTPS, managed by systemd, protected by daily backups, and accessible to your team around the clock. Reach out for assistance with any step.